IND320 Project Part 3

Reservoir seasonality and weather anomalies

This notebook extends Parts 1 and 2 with official NVE geographical regions, reproducible point sampling, ERA5 weather retrieval and four analyses: DCT/SPC, Local Outlier Factor, STL and a spectrogram.

GitHub: https://github.com/dano22222/dano22222

Streamlit application: https://dano22222-u9jpvjhwntlcuukcush85v.streamlit.app/

Screencast:

Completion rule: a single-point pilot or partial cache does not fulfil the required 1000-point regional mean. The next sections report which dataset was actually analysed.

1. Reproducible environment and configuration

The shared Python modules contain the production functions used by both this notebook and Streamlit. Imports do not trigger database writes or bulk downloads. The random seed is fixed during geographic preparation. Weather timestamps use UTC, avoiding daylight-saving gaps.

In [1]:
```
#locate the project from either the root or the notebooks folder
from pathlib import Path
import sys, json, importlib.metadata
ROOT = Path.cwd()
if not (ROOT / "src").exists():
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
from IPython.display import display, Markdown
from src.analysis import spc_analysis, lof_analysis, stl_analysis, spectrogr
from src.reservoir import weekly_series
from src.weather import load_region, estimate_download
REGION, START, END = "NO3", "2024-01-01", "2024-12-31"
print({name: importlib.metadata.version(name) for name in ["numpy", "pandas"
```

```
{'numpy': '2.4.6', 'pandas': '3.1.0', 'scipy': '1.17.1', 'scikit-learn': '1.
9.1', 'statsmodels': '0.15.0'}
```

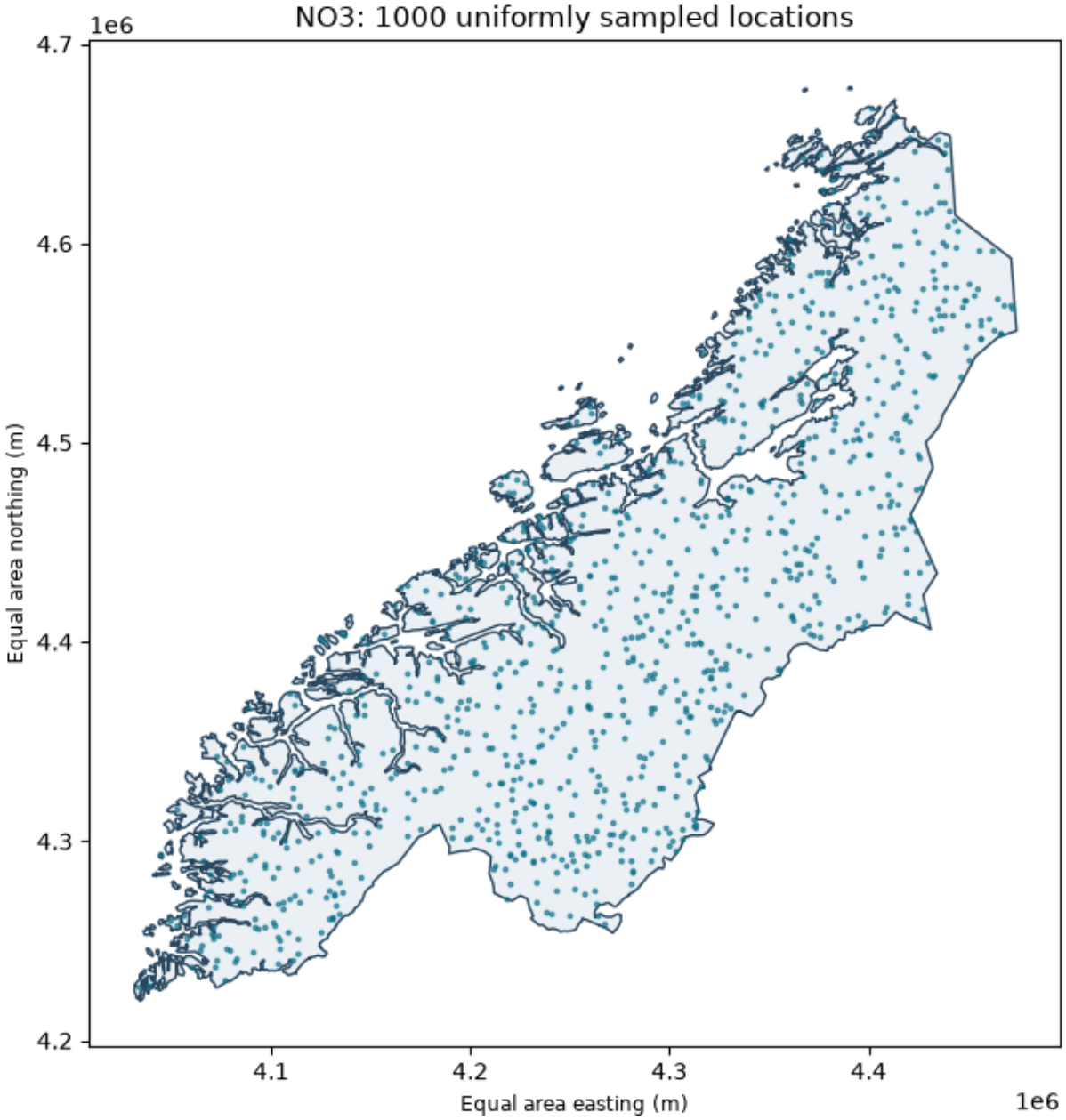
2. Geographical data and 1000 reproducible locations per region

The official NVE electricity and administrative polygon layers are retrieved programmatically from the services behind the NVE maps. Their original GeoJSON, exported shapefile bundles and source provenance are retained. Uniform rejection sampling takes place in equal-area EPSG:3035, then points are transformed to longitude/latitude.

The assignment combines east as RS+RØ and north as RM+RN. Current NVE metadata separately identifies VASS3 as Midt and VASS4 as Nord, whereas the historical reservoir response currently contains VASS1–3. We therefore retain the combined northern polygon as PRECIP_NORTH and map VASS3 to RM and VASS4 to RN. These administrative polygons are explicitly spatial proxies, **not exact hydrological catchments**. NO3 uses its actual electricity-area polygon. Coastal sampling and the API's land-cell choice also limit interpretation of regional averages.

In [2]:
```
#inspect saved official polygons and validate all sampled points
regions = gpd.read_file(ROOT / "data/regions/regions.geojson")
points = pd.read_csv(ROOT / "data/regions/points.csv")
display(points.groupby("region").size().rename("sampled locations").to_frame
assert points.groupby("region").size().eq(1000).all()
no3 = regions.loc[regions.region == REGION]
no3_points = points.loc[points.region == REGION]
locations = gpd.GeoSeries(gpd.points_from_xy(no3_points.longitude, no3_point
assert locations.within(no3.geometry.iloc[0]).all()
fig, ax = plt.subplots(figsize=(7, 7))
no3.to_crs(3035).plot(ax=ax, color="#edf3f8", edgecolor="#29485f")
locations.to_crs(3035).plot(ax=ax, markersize=3, color="#086e87", alpha=.6)
ax.set_title("NO3: 1000 uniformly sampled locations")
ax.set_xlabel("Equal area easting (m)"); ax.set_ylabel("Equal area northing
fig.tight_layout(); plt.show()
print((ROOT / "data/regions/provenance.json").read_text())
```

region	sampled locations
NATIONAL	1000
NO1	1000
NO2	1000
NO3	1000
NO4	1000
NO5	1000
PRECIP_NORTH	1000
VASS1	1000
VASS2	1000
VASS3	1000
VASS4	1000




```
{
  "sources": {
    "electricity": "https://gis3.nve.no/map/rest/services/Mapservices/Omradekonsesjoner/MapServer/8",
    "administrative": "https://gis3.nve.no/map/rest/services/Mapservices/Administrasjon/MapServer/0"
  },
  "seed": 320,
  "points_per_region": 1000,
  "sampling_crs": "EPSG:3035",
  "note": "VASS1=RS+R\u00d8, VASS2=RV, VASS3=RM, VASS4=RN; PRECIP_NORTH=RM+RN is the assignment combined north. All are administrative proxies, not exact catchments. Live NVE metadata distinguishes VASS3 mid and VASS4 north."
}
```


3. ERA5 download, resource estimate and data quality

`download_region(region, start, end, approved=...)` looks up the saved coordinate list and retrieves at least temperature and precipitation, explicitly selecting ERA5. A full leap year has 8784 hours. Data are checked for correct units, duplicates, complete time coverage and finite values. Each successful request is cached and can be reused after an interruption. A full regional result is only published after every sampled point is represented. The user approved the full NO3 download after receiving a resource estimate. No bulk download is triggered by running this notebook.

Source: Open-Meteo Historical Weather API, ERA5 / Copernicus, CC BY 4.0. This project computes spatial averages. Reanalysis and neighbouring grid cells are not independent weather-station measurements.

In [3]:
```
#report the estimate and select only a clearly identified available dataset
print(json.dumps(estimate_download(START, END), indent=2))
try:
    weather, weather_metadata = load_region(REGION, START, END)
    regional_complete = True
except FileNotFoundError:
    weather, weather_metadata = load_region(REGION, START, END, pilot=True)
    regional_complete = False
print(json.dumps(weather_metadata, indent=2))
print("complete regional experiment:", regional_complete)
if not regional_complete:
    display(Markdown("**INCOMPLETE REGIONAL EXPERIMENT: the following weathe
assert len(weather) == 8784 and weather.index.is_unique
assert np.isfinite(weather.to_numpy()).all()
display(weather.describe())
display(weather.head())
if regional_complete and (ROOT / "reports/weather_audit.json").exists():
    display(pd.Series(json.loads((ROOT / "reports/weather_audit.json").read_
```

```
{
  "points": 1000,
  "hours_per_point": 8784,
  "point_hours": 8784000,
  "values": 17568000,
  "float32_MB": 70.27,
  "estimated_transfer_MB": [
    211,
    703
  ],
  "estimated_api_call_units": 26142.9
}
{
  "region": "NO3",
  "start": "2024-01-01",
  "end": "2024-12-31",
  "points": 1000,
  "complete_1000": true,
  "model": "era5",
  "timezone": "UTC",
  "coordinates_hash": "e317ed1283d1",
  "source": "https://archive-api.open-meteo.com/v1/archive",
  "point_cache": "NO3_2024-01-01_2024-12-31_e317ed1283d1"
}
complete regional experiment: True
```


	temperature_2m	precipitation
count	8784.000000	8784.000000
mean	4.238217	0.197329
std	7.659723	0.283680
min	-16.737800	0.000000
25%	-0.817300	0.009700
50%	3.846300	0.084200
75%	10.041925	0.273600
max	24.328100	3.376700

timestamp	temperature_2m	precipitation
2024-01-01 00:00:00+00:00	-6.0342	0.0059
2024-01-01 01:00:00+00:00	-6.1923	0.0050
2024-01-01 02:00:00+00:00	-6.1487	0.0050
2024-01-01 03:00:00+00:00	-6.1830	0.0050
2024-01-01 04:00:00+00:00	-6.1134	0.0048

	verified result
region	NO3
points	1000
hours	8784
point_hours	8784000
scalar_values	17568000
missing_values	0
unique_returned_grid_cells	222
max_recomputed_mean_error	0.0
temperature_range_c	[-16.7378, 24.3281]
precipitation_range_mm_h	[0.0, 3.3767]
mean_annual_precipitation_mm	1733.337712
note	sampled points sharing era5 cells are spatiall...

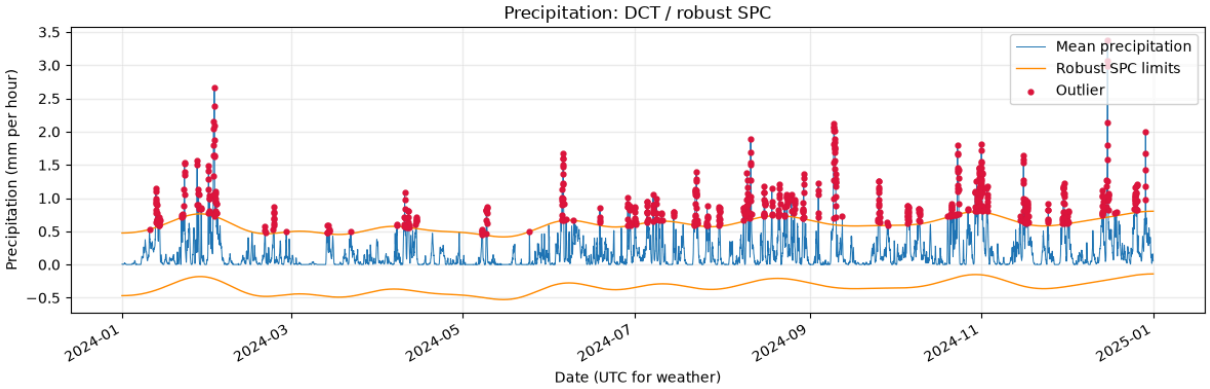
4. Precipitation outliers: DCT high pass and robust SPC

An orthonormal DCT-II represents the original hourly precipitation. Coefficient k has frequency k/(2 N Δt). Frequencies at or below 1/30 days form the smooth baseline; subtracting it gives the high-pass residuals. The entire year's residual median and Gaussian-consistent median absolute deviation (MAD) define robust limits at median ±3 MAD. Adding the baseline yields limits on the **original precipitation scale**. Only original precipitation, limits and contrasting outliers are plotted; residual/SATV values are not plotted.

The assignment's temperature wording in this precipitation section is interpreted as seasonally adjusted precipitation variations. Rainfall is skewed, autocorrelated and often zero, so these are descriptive control limits, not calibrated Gaussian probabilities. Zero MAD is explicitly reported. Negative lower limits are retained to show the mathematical limits and do not imply physically negative rain. Cutoff and sigma multiplier are configurable.

In [4]:
```
#analyse precipitation and display the actual flagged observations
CUTOFF_DAYS, N_SIGMA = 30, 3
fig, spc_table, spc_summary = spc_analysis(weather.precipitation, CUTOFF_DAY
plt.show()
print(spc_summary)
display(spc_table.loc[spc_table.outlier].head(20))
(ROOT / "reports/figures").mkdir(parents=True, exist_ok=True)
```

```
fig.savefig(ROOT / "reports/figures/precipitation_spc.png", dpi=160, bbox_ir
spc_table.to_csv(ROOT / "reports/spc_results.csv")
```



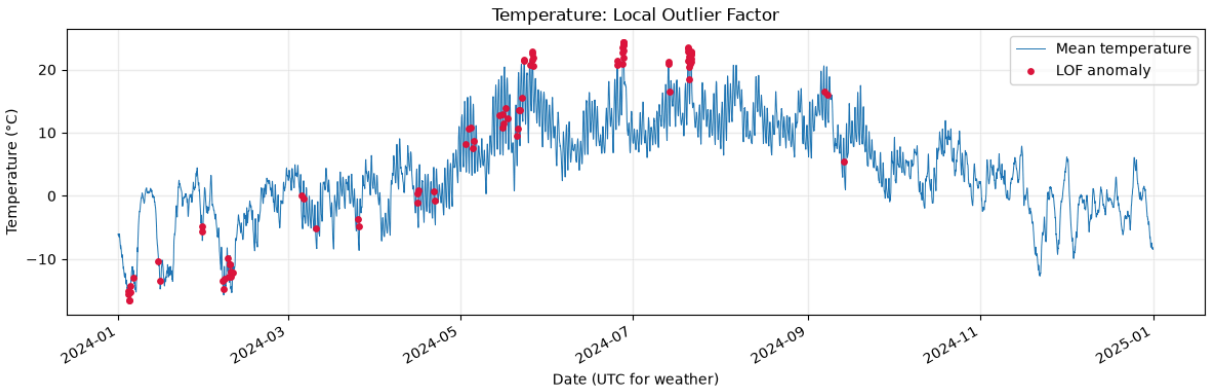

```
{'observations': 8784, 'outliers': 633, 'fraction': 0.07206284153005464, 'ro
bust_sigma': 0.15724150253350086, 'zero_scale': False}
```

timestamp	value	baseline	lower	upper	outlier
2024-01-10 17:00:00+00:00	0.5295	0.123626	-0.414865	0.528584	True
2024-01-12 21:00:00+00:00	0.6326	0.151383	-0.387108	0.556341	True
2024-01-12 22:00:00+00:00	0.7686	0.151971	-0.386520	0.556929	True
2024-01-12 23:00:00+00:00	0.9322	0.152560	-0.385930	0.557519	True
2024-01-13 00:00:00+00:00	1.1437	0.153152	-0.385339	0.558110	True
2024-01-13 01:00:00+00:00	1.1075	0.153746	-0.384745	0.558704	True
2024-01-13 02:00:00+00:00	0.9133	0.154341	-0.384149	0.559300	True
2024-01-13 03:00:00+00:00	0.8162	0.154939	-0.383552	0.559897	True
2024-01-13 04:00:00+00:00	0.9598	0.155538	-0.382953	0.560496	True
2024-01-13 05:00:00+00:00	0.9916	0.156139	-0.382352	0.561097	True
2024-01-13 06:00:00+00:00	0.8948	0.156742	-0.381749	0.561700	True
2024-01-13 07:00:00+00:00	0.7784	0.157347	-0.381144	0.562305	True
2024-01-13 08:00:00+00:00	0.6821	0.157954	-0.380537	0.562912	True
2024-01-13 09:00:00+00:00	0.6286	0.158562	-0.379929	0.563520	True
2024-01-13 10:00:00+00:00	0.6152	0.159173	-0.379318	0.564131	True
2024-01-13 11:00:00+00:00	0.6741	0.159785	-0.378706	0.564743	True
2024-01-13 12:00:00+00:00	0.6723	0.160399	-0.378092	0.565357	True
2024-01-13 13:00:00+00:00	0.6584	0.161014	-0.377476	0.565973	True
2024-01-13 14:00:00+00:00	0.6603	0.161632	-0.376859	0.566590	True
2024-01-13 15:00:00+00:00	0.6554	0.162251	-0.376239	0.567210	True

5. Temperature anomalies: Local Outlier Factor

LOF compares local neighbour densities using two standardized features: temperature and its hourly change. The first change is defined as zero. Defaults are 48 neighbours and 1% contamination. This can highlight unusual temperatures or rapid changes relative to the data's density. Contamination is a modelling assumption; detected anomalies are not necessarily errors. The model is descriptive and has no training/test claim or causal interpretation.

In [5]:
```
#identify temperature anomalies using the same complete hourly time index
fig, lof_table, lof_summary = lof_analysis(weather.temperature_2m, contamina
plt.show()
print(lof_summary)
display(lof_table.loc[lof_table.outlier].sort_values("lof_score", ascending=
fig.savefig(ROOT / "reports/figures/temperature_lof.png", dpi=160, bbox_inch
lof_table.to_csv(ROOT / "reports/lof_results.csv")
```




```
{'observations': 8784, 'outliers': 88, 'fraction': 0.010018214936247723}
```

timestamp	value	lof_score	outlier
2024-02-09 16:00:00+00:00	-11.9860	2.124686	True
2024-02-07 16:00:00+00:00	-13.1029	2.072824	True
2024-02-09 15:00:00+00:00	-10.8009	2.069047	True
2024-05-16 20:00:00+00:00	13.9061	2.032413	True
2024-03-25 18:00:00+00:00	-3.7058	1.882231	True
2024-04-21 19:00:00+00:00	0.7603	1.872662	True
2024-05-15 20:00:00+00:00	12.9215	1.870787	True
2024-05-22 06:00:00+00:00	13.6301	1.803361	True
2024-05-21 06:00:00+00:00	10.6111	1.778859	True
2024-06-27 17:00:00+00:00	23.8760	1.707164	True
2024-05-04 19:00:00+00:00	10.8336	1.695178	True
2024-06-27 16:00:00+00:00	24.3042	1.692674	True
2024-06-27 15:00:00+00:00	24.2909	1.692283	True
2024-05-20 20:00:00+00:00	9.5224	1.676166	True
2024-04-15 19:00:00+00:00	0.4440	1.659200	True
2024-07-13 20:00:00+00:00	16.4396	1.638075	True
2024-07-20 16:00:00+00:00	23.3338	1.637026	True
2024-07-20 17:00:00+00:00	22.8055	1.601056	True
2024-04-15 20:00:00+00:00	-0.9991	1.599927	True
2024-04-21 20:00:00+00:00	-0.6988	1.585090	True

Parameter sensitivity

Outlier counts depend on modelling choices. The following checks vary the DCT cutoff and the assumed LOF contamination while holding other defaults fixed. More flags do not establish worse data quality; they show sensitivity to the definition of an unusual observation.

In [6]:
```
#compare parameter choices without displaying additional duplicate figures
sensitivity = []
for cutoff in [7, 30, 90]:
    figure, table, summary = spc_analysis(weather.precipitation, cutoff_days
    plt.close(figure)
    sensitivity.append({"method": "spc", "parameter": f"cutoff {cutoff} days
for fraction in [.005, .01, .02]:
    figure, table, summary = lof_analysis(weather.temperature_2m, contaminat
```

```
    plt.close(figure)
    sensitivity.append({"method": "lof", "parameter": f"contamination {100*f
display(pd.DataFrame(sensitivity))
```

	method	parameter	flagged
0	spc	cutoff 7 days	640
1	spc	cutoff 30 days	633
2	spc	cutoff 90 days	725
3	lof	contamination 0.5%	44
4	lof	contamination 1%	88
5	lof	contamination 2%	176

6. Reservoir filling: prepare a regular weekly series

Use one reservoir area's filling fraction, not an average of overlapping national and regional statistics. The included NVE snapshot makes the analysis reproducible offline; the Streamlit app continues to retrieve current data from NVE. Missing weeks and duplicate timestamps are rejected rather than silently interpolated. Several years are needed to distinguish annual seasonality from trend.

In [7]:
```
#select the same no3 electricity area and inspect weekly coverage
reservoir = pd.read_parquet(ROOT / "data/reservoir.parquet")
filling = weekly_series(reservoir, area_type="EL", area=3)
print("weekly observations:", len(filling))
print("time interval:", filling.index.min(), filling.index.max())
print("step sizes:", filling.index.to_series().diff().dropna().value_counts(
display(filling.describe())
```

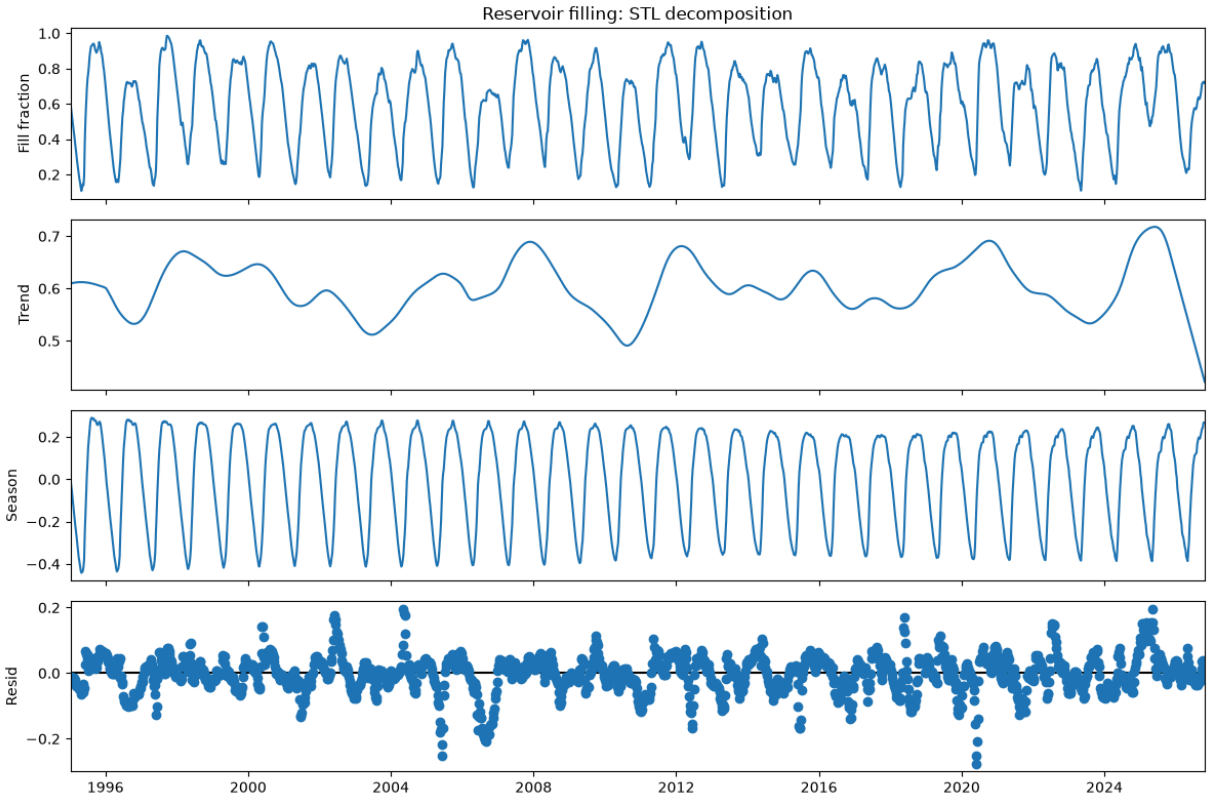
```
weekly observations: 1660
time interval: 1995-01-08 00:00:00 2026-10-25 00:00:00
step sizes: {Timedelta('7 days 00:00:00'): 1659}
count    1660.000000
mean        0.597635
std         0.236924
min         0.107109
25%         0.391493
50%         0.640104
75%         0.801169
max         0.985538
Name: fill_level, dtype: float64
```

7. Seasonal trend decomposition using LOESS

STL separates observed filling fraction into trend, seasonal component and residual. Use annual period 52 weekly observations, seasonal smoother 13, trend smoother 105 and robust fitting by default. Smoothers must be odd; trend must exceed the seasonal

period. A 52-week year approximates the true calendar year. Robust fitting reduces the influence of exceptional weeks without proving that they are erroneous.

In [8]:
```
#decompose the entire regular weekly series and verify reconstruction
fig, stl_result = stl_analysis(filling, seasonal=13, trend=105, robust=True)
plt.show()
assert np.allclose(stl_result.trend + stl_result.seasonal + stl_result.resid
fig.savefig(ROOT / "reports/figures/reservoir_stl.png", dpi=160, bbox_inches
print("seasonal range:", float(stl_result.seasonal.max() - stl_result.seasor
print("residual standard deviation:", float(stl_result.resid.std()))
```



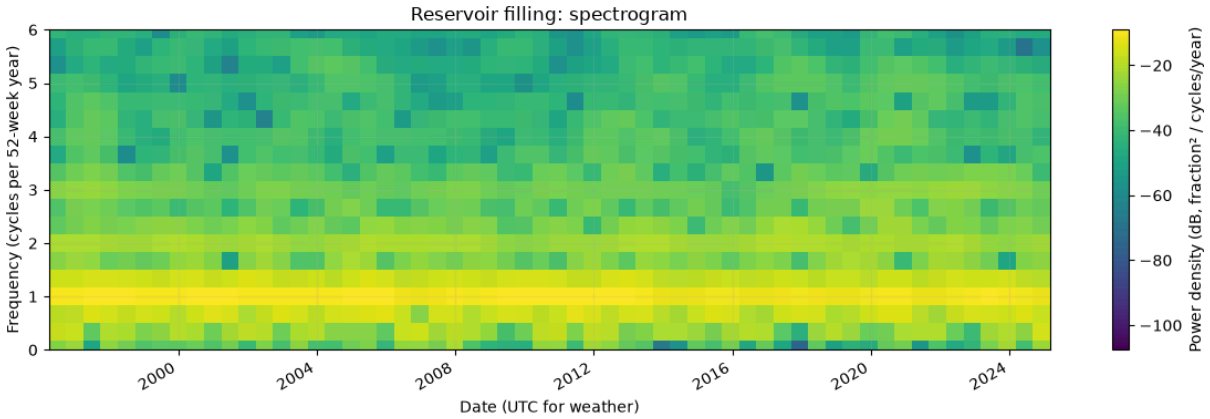

```
seasonal range: 0.732700354123887
residual standard deviation: 0.05661886825485383
```

8. Spectrogram of the same filling series

A Hann-window spectrogram uses 156 weekly samples per window with 130 samples overlap. Each segment is linearly detrended. Frequencies are cycles per 52-week year; 1 cycle/year represents annual variation. The frequency resolution is 52/156 = 1/3 cycles/year. Longer windows improve frequency resolution while reducing temporal localisation. Colour represents log power density; a small floor is used only to display logarithms. The plot focuses on 0–6 cycles/year; the full computed spectrum extends to the weekly Nyquist frequency of 26 cycles/year. Only complete windows are shown, so their centres do not reach the endpoints.

In [9]:
```
#use exactly the same weekly filling series as the stl analysis
fig, spectrum = spectrogram_analysis(filling, window=156, overlap=130)
```

```
plt.show()
fig.savefig(ROOT / "reports/figures/reservoir_spectrogram.png", dpi=160, bbc
nonzero = spectrum["frequency"] > 0
frequency = spectrum["frequency"][nonzero]
mean_power = spectrum["power"][nonzero].mean(axis=1)
print("dominant nonzero frequency:", float(frequency[mean_power.argmax()]),
```




```
dominant nonzero frequency: 1.0 cycles per year
```

9. Function tests and scientific validation

The test suite checks known annual periodicity, STL reconstruction and seasonal recovery, injected SPC/LOF anomalies, constant signals, invalid parameters, missing timestamps, deterministic area sampling, polygon containment, download approval, cache reuse and rate-limit handling. Streamlit tests exercise date filtering, comparison controls and the new analysis pages. These tests complement scientific interpretation; they do not prove all real-weather flags are meaningful.

In [10]:
```
#run the local test suite without making external database writes
import subprocess
check = subprocess.run([sys.executable, "-m", "pytest", "-q"], cwd=ROOT, cap
print(check.stdout)
if check.returncode:
    print(check.stderr)
assert check.returncode == 0
```

```
.....................................................................  [ 9
4%]
....                                                                   [10
0%]
============================== warnings summary ===========================
====
tests/test_streamlit 2.py::test_plot_default_and_comparison
tests/test_streamlit 2.py::test_existing_electricity_page_with_database_fixt
ure
tests/test_streamlit.py::test_plot_default_and_comparison
tests/test_streamlit.py::test_existing_electricity_page_with_database_fixtur
e
  /Users/daniaromar/Desktop/Universitetet/7. Høst 2026/IND320/Project/.venv/
lib/python3.11/site-packages/streamlit/elements/lib/built_in_chart_utils.py:
620: Pandas4Warning: The inplace keyword in DataFrame.reset_index is depreca
ted and will be removed in a future version (PDEP-8).
  See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-in
place for more details.
    df.reset_index(inplace=True)  # noqa: PD002

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
76 passed, 4 warnings in 12.45s
```

10. Interpretation of the actual results

In [11]:
```
#summarize computed results while keeping completion status explicit
status = "1000 point regional mean" if regional_complete else "single point
display(Markdown(f"""The weather results above use **{status}**. SPC flags *

The reservoir series contains **{len(filling)} weekly observations**. STL re

Regional averaging smooths local extremes, and repeated ERA5 grid cells make
```

The weather results above use **1000 point regional mean**. SPC flags **633** of 8784 hourly observations with a 30 day baseline and three robust sigmas. LOF flags **88** observations with the assumed 1% contamination. These methods answer different questions: SPC measures departures from a smooth precipitation baseline, while LOF identifies locally unusual temperature states or changes. Neither establishes a data error or physical cause.

The reservoir series contains **1660 weekly observations**. STL reconstruction was verified numerically. The strongest nonzero average spectral frequency is **1.000 cycles per year**. This describes a frequency component rather than a causal driver. Inspect the decomposition and changes in spectral power alongside the original data.

Regional averaging smooths local extremes, and repeated ERA5 grid cells make sampled points spatially dependent. Results depend on cutoff, neighbours, assumed contamination and smoothing settings; these are exposed in Streamlit for sensitivity analysis.

11. Streamlit integration and Part 2 feedback

The existing Streamlit URL is retained. The Reservoir Data date interval filters overview sparklines, detailed rows and CSV export together. Shared area selection connects reservoir and weather pages. Optional comparisons and separate plots for different units improve readability. New pages provide `st.tabs()` for STL/Spectrogram and SPC/LOF, with user controls, figures and outlier summaries. The existing electricity page, MongoDB secrets handling and NVE API connection remain available. A single-point request is explicit; bulk weather jobs run locally with approval and resumable caching.

12. Development log

Part 3 extends the existing reservoir and electricity application with regional weather data and time series analysis. The first step was to inspect the Part 1 and Part 2 notebooks, Python files and Git history. The existing work already included direct NVE access, an ENTSO-E processing pipeline using Spark and Cassandra, and a MongoDB-backed electricity page. These components were retained. A separate Part 3 development branch was created, and files changed during this implementation were backed up. The remote history confirmed the earlier Part 2 merge into main.

The geographical workflow retrieves official NVE polygons, retains their provenance and exports shapefiles. Locations are sampled reproducibly in an equal-area projection before conversion to latitude and longitude. Current NVE metadata distinguishes middle and northern VASS areas, which differs from the assignment's combined north wording. Both the mapping and its limitations are documented instead of treating administrative boundaries as exact catchments.

Weather retrieval uses the historical Open-Meteo endpoint with the ERA5 model selected explicitly. The downloader checks units, hourly coverage, duplicates and missing values. Each successful point is cached separately, allowing interrupted downloads to resume. Resource use was estimated before the full NO3 download was approved. After validating a single-point pilot, the complete NO3 experiment was downloaded: 1000 locations with 8784 hourly observations each. An hourly quota pause was respected, and the last 43 locations were retrieved after the next hourly reset. The regional mean was independently checked against all cached point series.

Four analysis functions were implemented for reuse in the notebook and Streamlit. Precipitation outliers use a DCT baseline and robust residual control limits. Temperature anomalies use Local Outlier Factor on standardized temperature and hourly changes. Weekly reservoir filling is analysed with robust STL and a Hann-window spectrogram. Synthetic signals and injected anomalies test meaningful scientific properties, including seasonal recovery, reconstruction and the expected annual spectral peak.

The Streamlit work incorporates feedback from Part 2. The Reservoir Data page now filters its overview, detailed dataset and export by the same date interval. Shared area

selection connects reservoir and weather analyses. Optional area comparison and separate variable panels improve readability. New tabbed pages expose analysis parameters, plots and summaries. The original app address is retained, and bulk weather retrieval is controlled locally rather than triggered by ordinary page interactions.

Finally, the project gains organised notebook and report folders, installation instructions, a delivery checklist and a screencast outline. Automated checks cover data preparation, numerical analyses, downloader behaviour and Streamlit interactions. All 38 tests pass, the complete regional results have been verified, and Part 3 has been merged into main and deployed at the original address.

13. AI usage

OpenAI ChatGPT served as a supporting tool for implementation planning, API documentation research, Streamlit integration, and notebook preparation in Part 3. It was also used to assist with Python programming, as well as to refine the overall structure and documentation of the Jupyter Notebook and Streamlit application.

During Part 3, ChatGPT provided guidance when testing code locally using numerical checks, deterministic synthetic signals, real NVE data, and an ERA5 pilot. It assisted in structuring test logs to distinguish completed checks from pending external tasks, and offered advice on how to preserve the existing Spark, Cassandra, and MongoDB pipeline without rerunning database writes ensuring historical outputs were correctly documented rather than presented as newly verified results.

Every suggestion, API response, and data transformation was critically reviewed, adapted, and manually tested before being incorporated. This included verifying local test results and ensuring that the overall pipeline functionality remained intact.

ChatGPT provided helpful input and feedback throughout the problem-solving process, but all final decisions, code implementation, testing, and evaluation of results remained entirely my own responsibility.

14. Submission and sources

Review the development log and geographic assumptions, run this notebook on the complete regional dataset, inspect the PDF export and record the five minute video. The repository must show a merge from the Part 3 development branch into main. Update the existing app rather than creating a new URL. The video link must be accessible to reviewers.

- NVE reservoir source: https://www.nve.no/energi/analyser-og-statistikk/om-magasinstatistikken/

- NVE area metadata: https://biapi.nve.no/magasinstatistikk/api/Magasinstatistikk/HentOmr%C3%A5der
- Electricity polygons: https://gis3.nve.no/map/rest/services/Mapservices/Omradekonsesjoner/MapServer/8
- Administrative polygons: https://gis3.nve.no/map/rest/services/Mapservices/Administrasjon/MapServer/0
- ERA5 API and attribution: https://open-meteo.com/en/docs/historical-weather-api
- API quotas: https://open-meteo.com/en/pricing

The full methodological record is in `reports/ASSUMPTIONS.md`.

Appendix: shared function implementations

These are the actual functions imported and tested above. Printing their source keeps the PDF reviewable without duplicating production logic in the notebook. Their imports and supporting helpers are maintained in the accompanying repository modules. Printing source does not launch any extra downloads.

In [12]:
```
#include the actual reusable implementations in the executed report
import inspect
from src.weather import download_point, download_region
from src.geography import sample_polygon
for function in [sample_polygon, download_point, download_region, spc_analys
    print(function.__name__)
    print(inspect.getsource(function))
```

```
sample_polygon
def sample_polygon(geometry, count=1000, seed=320, original_wgs84=None):
    """uniform area in epsg:3035 (equal area), then convert to lon/lat."""
    if count<1 or geometry.is_empty:raise ValueError('Nonempty polygon and p
ositive count required.')
    rng=np.random.default_rng(seed);xmin,ymin,xmax,ymax=geometry.bounds
    points=[]
    for _ in range(10000):
        x=rng.uniform(xmin,xmax,max(1000,count*2));y=rng.uniform(ymin,ymax,l
en(x))
        inside=contains_xy(geometry,x,y)
        accepted_x,accepted_y=x[inside],y[inside]
        if original_wgs84 is not None:
            #verify containment after transformation because polygon edges c
hange projection
            candidate=gpd.GeoSeries(gpd.points_from_xy(accepted_x,accepted_
y),crs=3035).to_crs(4326)
            valid=contains_xy(original_wgs84,candidate.x.to_numpy(),candidat
e.y.to_numpy())
            accepted_x,accepted_y=accepted_x[valid],accepted_y[valid]
        points.extend(zip(accepted_x,accepted_y))
        if len(points)>=count:break
    if len(points)<count:raise ValueError('Sampling failed: extremely sparse
polygon.')
    xy=np.asarray(points[:count])
    return gpd.GeoSeries(gpd.points_from_xy(xy[:,0],xy[:,1]),crs=3035).to_cr
s(4326)

download_point
def download_point(latitude,longitude,start,end,session=None):
    session=session or requests.Session()
    response=None
    for attempt in range(4):
        response=session.get(URL,params={'latitude':float(latitude),'longitu
de':float(longitude),
            'start_date':str(start),'end_date':str(end),'hourly':','.join(VA
RIABLES),
            'models':'era5','timezone':'UTC','cell_selection':'land'},timeou
t=120)
        if response.status_code==429:
            try:
                reason = response.json().get('reason', 'quota reached')
            except ValueError:
                reason = 'quota reached'
            raise WeatherRateLimit(f'Open-Meteo: {reason}. Cached points are
safe; resume later.')
        if response.status_code>=500 and attempt<3:
            time.sleep(2**attempt);continue
        response.raise_for_status();break
    payload=response.json()
    if payload.get('error'):raise ValueError(payload.get('reason','Open-Mete
o error'))
    if payload.get('hourly_units',{}).get('temperature_2m')!='°C' or payloa
d.get('hourly_units',{}).get('precipitation')!='mm':
        raise ValueError('Unexpected weather units.')
    df=pd.DataFrame(payload['hourly']).rename(columns={'time':'timestamp'})
```

```
    df['timestamp']=pd.to_datetime(df.timestamp,utc=True)
    df=df.set_index('timestamp')[VARIABLES].astype('float32')
    validate_hourly(df,start,end)
    return df, {k:payload.get(k) for k in ['latitude','longitude','elevatio
n','timezone','hourly_units']}

download_region
def download_region(region,start,end,approved=False,max_points=None,delay=2.
5):
    """lookup fixed coordinates, cache each point, and publish mean only if
complete.

    year long calls have weighted quota units. stop on 429; reuse cached fil
es
    on the next invocation. never round/deduplicate coordinates: elevation
    adjustment and nearest grid selection can differ between sampled points.
    """
    points=pd.read_csv(ROOT/'data/regions/points.csv')
    points=points.loc[points.region==region].sort_values('point_id')
    if points.empty:raise ValueError(f'Unknown region {region}')
    if max_points is not None:points=points.head(max_points)
    if len(points)>1 and not approved:raise PermissionError('Bulk weather do
wnload requires explicit approval.')
    fingerprint=hashlib.sha256(points.to_csv(index=False).encode()).hexdiges
t()[:12]
    folder=WEATHER_DIR/f'{region}_{start}_{end}_{fingerprint}'
    folder.mkdir(parents=True,exist_ok=True)
    total=np.zeros((len(expected_hours(start,end)),2),dtype='float64')
    session=requests.Session();downloaded=0
    for row in points.itertuples():
        path=folder/f'point_{row.point_id:04d}.parquet'
        if path.exists():
            df=validate_hourly(pd.read_parquet(path),start,end)
        else:
            df,metadata=download_point(row.latitude,row.longitude,start,end,
session)
            temporary=path.with_suffix('.tmp');df.to_parquet(temporary);temp
orary.replace(path)
            (folder/f'point_{row.point_id:04d}.json').write_text(json.dumps
({
                'requested_latitude':row.latitude,'requested_longitude':row.
longitude,
                'source':URL,'model':'era5',**metadata},indent=2))
            downloaded+=1
            if downloaded%10==0:print(f'{region}: {downloaded} new points; c
urrent ID {row.point_id}',flush=True)
            time.sleep(delay)
        total+=df[VARIABLES].to_numpy()
    result=pd.DataFrame(total/len(points),index=expected_hours(start,end),co
lumns=VARIABLES)
    result.index.name='timestamp'
    label='mean' if len(points)==1000 else f'pilot_{len(points)}'
    target=WEATHER_DIR/f'{region}_{start}_{end}_{label}.parquet'
    result.to_parquet(target)
    target.with_suffix('.json').write_text(json.dumps({'region':region,'star
t':start,'end':end,
```

```
        'points':len(points),'complete_1000':len(points)==1000,'model':'era
5','timezone':'UTC',
        'coordinates_hash':fingerprint,'source':URL,'point_cache':folder.nam
e},indent=2))
    return result

spc_analysis
def spc_analysis(series, cutoff_days=30., n_sigma=3.):
    """dct ii high pass; median and gaussian consistent mad over the entire
input.

    frequencies are k/(2*n*dt). keep periods >= cutoff_days in the baseline.
    plot the baseline plus the residual median, with limits n_sigma times ma
d away.
    these are descriptive robust control limits, not calibrated rain probabi
lities.
    """
    s = clean_series(series)
    if s.index[1] - s.index[0] != pd.Timedelta(hours=1):
        raise ValueError("spc expects hourly precipitation observations.")
    if cutoff_days <= 0 or not np.isfinite(cutoff_days) or n_sigma <= 0 or n
ot np.isfinite(n_sigma):
        raise ValueError('Cutoff and sigma multiplier must be finite and pos
itive.')
    dt_days = (s.index[1] - s.index[0]).total_seconds()/86400
    coefficients = dct(s.values, norm='ortho')
    frequencies = np.arange(len(s))/(2*len(s)*dt_days)
    coefficients[frequencies > 1/cutoff_days] = 0
    baseline = idct(coefficients, norm='ortho')
    residual = s.values - baseline
    centre = np.median(residual)
    scale = float(median_abs_deviation(residual, scale='normal'))
    #a zero robust scale is possible with sparse rain. do not substitute non
robust std.
    tolerance = np.finfo(float).eps * max(1., float(np.max(np.abs(s)))) * 10
0
    if scale < tolerance:
        scale = 0.
    lower = baseline + centre - n_sigma*scale
    upper = baseline + centre + n_sigma*scale
    flags = (s.values < lower-tolerance) | (s.values > upper+tolerance)
    table = pd.DataFrame({'value':s, 'baseline':baseline, 'lower':lower,
                          'upper':upper, 'outlier':flags})
    fig, ax = plt.subplots(figsize=(12,4))
    ax.plot(s.index, s.values, lw=.7, label='Mean precipitation')
    ax.plot(s.index, lower, color='darkorange', lw=1, label='Robust SPC limi
ts')
    ax.plot(s.index, upper, color='darkorange', lw=1)
    ax.scatter(s.index[flags], s.values[flags], c='crimson', s=12, label='Ou
tlier', zorder=3)
    style(ax, 'Precipitation: DCT / robust SPC', 'Precipitation (mm per hou
r)')
    ax.legend(loc='upper right'); fig.tight_layout()
    return fig, table, {'observations':len(s), 'outliers':int(flags.sum()),
                        'fraction':float(flags.mean()), 'robust_sigma':scal
e,
```

```
                    'zero_scale':scale == 0}

lof_analysis
def lof_analysis(series, contamination=.01, n_neighbors=48):
    """lof on standardized [temperature, 1 hour change] to detect unusual st
ates.

    the first change is set to zero; this boundary convention is explicit.
    lof is descriptive: the requested fraction is a modelling assumption.
    """
    s = clean_series(series)
    if s.index[1] - s.index[0] != pd.Timedelta(hours=1):
        raise ValueError("lof expects hourly temperature observations.")
    if not 0 < contamination <= .5 or not 2 <= n_neighbors < len(s):
        raise ValueError('Use 0 < contamination <= .5 and 2 <= neighbors <
N.')
    features = np.column_stack([s.values, s.diff().fillna(0).values])
    features = StandardScaler().fit_transform(features)
    model = LocalOutlierFactor(n_neighbors=int(n_neighbors), contamination=c
ontamination)
    flags = model.fit_predict(features) == -1
    table = pd.DataFrame({'value':s, 'lof_score':-model.negative_outlier_fac
tor_, 'outlier':flags})
    fig, ax = plt.subplots(figsize=(12,4))
    ax.plot(s.index, s.values, lw=.7, label='Mean temperature')
    ax.scatter(s.index[flags], s.values[flags], s=15, color='crimson', label
='LOF anomaly', zorder=3)
    style(ax, 'Temperature: Local Outlier Factor', 'Temperature (°C)')
    ax.legend();fig.tight_layout()
    return fig, table, {'observations':len(s),'outliers':int(flags.sum()),'f
raction':float(flags.mean())}

stl_analysis
def stl_analysis(series, seasonal=13, trend=105, robust=True, period=52):
    """annual seasonality approximated by 52 weekly samples; need >=2 cycle
s."""
    if period < 2 or seasonal < 3 or seasonal % 2 != 1 or trend <= period or
trend % 2 != 1:
        raise ValueError('Seasonal must be odd >=3; trend odd > period; peri
od >=2.')
    s = clean_series(series, minimum=2*period)
    if s.index[1] - s.index[0] != pd.Timedelta(days=7):
        raise ValueError("stl expects weekly reservoir observations.")
    result = STL(s, period=period, seasonal=seasonal, trend=trend, robust=ro
bust).fit()
    fig = result.plot();fig.set_size_inches(12,8)
    fig.axes[0].set_title('Reservoir filling: STL decomposition')
    fig.axes[0].set_ylabel('Fill fraction');fig.tight_layout()
    return fig, result

spectrogram_analysis
def spectrogram_analysis(series, window=156, overlap=130):
    """hann window power density, cycles/year; remove overall linear trend.

    weekly sampling is assumed and checked. resolution = 52/window cycles/ye
ar.
```

```
    """
    if not isinstance(window, (int,np.integer)) or not isinstance(overlap,(i
nt,np.integer)):
        raise ValueError('Window and overlap must be integers.')
    if not 0 <= overlap < window or window < 8:
        raise ValueError('Need window >=8 and 0 <= overlap < window.')
    s = clean_series(series, minimum=window)
    step = (s.index[1]-s.index[0]).total_seconds()/86400
    if not np.isclose(step,7):
        raise ValueError('Spectrogram expects weekly data.')
    f,t,power = spectrogram(s.values, fs=52., window='hann', nperseg=window,
                            noverlap=overlap, detrend='linear', scaling='den
sity')
    dates = s.index[0] + pd.to_timedelta(t*7*52,unit='D')
    fig, ax = plt.subplots(figsize=(12,4))
    mesh = ax.pcolormesh(dates, f, 10*np.log10(np.maximum(power,1e-15)), sha
ding='auto', cmap='viridis')
    ax.set_ylim(0,6)
    style(ax,'Reservoir filling: spectrogram','Frequency (cycles per 52-week
year)')
    fig.colorbar(mesh,ax=ax,label='Power density (dB, fraction² / cycles/yea
r)')
    fig.tight_layout()
    return fig, {'frequency':f,'time_years':t,'power':power}
```